
BRAIN OF THE DIGITAL TWIN

*System Architecture and Implementation of an Agentic Digital Twin Pipeline
using LangGraph, Gaussian Process Regression, and Bayesian Active Learning*

A detailed technical documentation of the five-stage autonomous pipeline for generating synthetic 3D-printer telemetry data, covering the State Registry, the Idempotent Ingestion Engine, and the Gaussian Process Surrogate Model.

Pipeline	Agentic Digital Twin (FDM / Ultimaker S5)
Orchestrator	LangGraph (Directed Acyclic Graph)
Surrogate	Gaussian Process Regressor (GPR)
Loop Method	Bayesian Active Learning (BAL)
Target Volume	10,000 synthetic rows

June 9, 2026

CONTENTS

1	Brain of the Digital Twin — Pipeline Overview	3
2	System Architecture: The State Telemetry Registry (State.py)	4
2.1	What Is the Role of This File?	4
2.2	Code Overview and Explanation	5
2.2.1	The Contract: <code>TypedDict</code>	5
2.2.2	The Pointer-Based Memory Architecture	5
2.2.3	The Active Learning Loop Breakers	5
2.2.4	Important Clarifications	6
3	System Architecture: Receiving Dock (stage_01_ingestion.py)	7
3.1	What Does This Node Do?	8
3.2	What Is the Need for This Node?	8
3.3	Code Overview and Explanation	8
3.3.1	The Custom Exceptions (Fail-Fast Methodology)	8
3.3.2	The Engine Overview (Encapsulation)	9
3.3.3	The Internal Checks (Private Methods)	9
3.3.4	The Smart Caching	10
3.3.5	The LangGraph Translator (The Node Interface)	10
3.3.6	Important Clarifications	12
3.3.7	Coding Implementation Notes	14
4	System Architecture: Quality Control Sorter (stage_02_cleaning.py)	15
4.1	What Does This Node Do?	15
4.2	What Is the Need for This Node?	15
4.3	Code Overview and Explanation	16
4.3.1	The Custom Exceptions (Fail-Fast Methodology)	16
4.3.2	The Engine Overview (Encapsulation)	16
4.3.3	Schema Normalization	16
4.3.4	Target Isolation (Boolean Masking)	17
4.3.5	Anomaly Mitigation (Mathematical Imputation)	17
4.3.6	The LangGraph Translator (The Node Interface)	17
4.3.7	Important Clarifications	18
5	System Architecture: Blindfolded Surveyor (stage_03_surrogate.py)	19
5.1	The Concept: What Is a Gaussian Process Regressor?	19

5.2	Why Is This Required for the 3D Printer?	19
5.3	Understanding GPR: Gaussian Process Regression	20
5.3.1	Non-Parametric Machine Learning Model	20
5.3.2	Gaussian Process Regression (GPR)	20
5.3.3	Gaussian (Normal) Distribution	21
5.3.4	Joint Probability Density Function (Multivariate Gaussian in GP)	22
5.3.5	The Posterior Distribution	23
5.4	Code Overview and Explanation	25
5.4.1	Library Imports and Error Handling	25
5.4.2	Building the Mapper (Class Initialization and Hyperparameters)	26
5.4.3	Dynamic Configuration (<code>_ingest_assets</code>)	26
5.4.4	Mathematical Guardrails (<code>_construct_kernel</code>)	27
5.4.5	Core Math and Validation (<code>execute_training_sequence</code>)	28
5.4.6	Serialization	30
5.4.7	The LangGraph Translator (<code>fit_surrogate_model</code>)	31
5.4.8	Understanding the Kernel and Its Parameters	32
5.4.9	Important Clarifications	34
6	System Architecture: Quality Assurance Gate (<code>stage_05_validation.py</code>)	35
6.1	Code Overview and Explanation	35
6.1.1	Imports and Exception Handling	35
6.1.2	Initialization and Physical Constraints (<code>__init__</code>)	36
6.1.3	Metadata Purging (<code>_initialize_environment</code>)	36
6.1.4	Structural Integrity (<code>_evaluate_structural_integrity</code>)	36
6.1.5	Floating-Point Tolerance (<code>_evaluate_physical_boundaries</code>)	37
6.1.6	Surrogate Confidence Gate (<code>_evaluate_surrogate_confidence</code>)	37
6.1.7	LangGraph Orchestration (<code>enforce_physics</code>)	38
6.2	Important Clarifications	38

BRAIN OF THE DIGITAL TWIN — PIPELINE OVERVIEW

The execution flow of the Agentic Digital Twin is managed by a **DAG architecture**. It works as a pipeline where different nodes are connected in a fixed workflow (DAG). The orchestrator controls:

- Node execution
- State passing
- Retries
- Loops
- Failure handling
- Stopping conditions

LangGraph is a state graph, which operates like a factory assembly line. There are five workers in our system at different stations on the line:

Stage 01 — The Ingestion Worker (`stage_01_ingestion.py`). *Job: The raw materials receiver.*

This worker goes to the remote warehouse (which in our case is Kaggle), securely downloads the raw 3D printer dataset, saves it to the local hard drive, and writes the exact file path on the dictionary (clipboard).

Stage 02 — The Sanitization Worker (`stage_02_cleaning.py`). *Job: The quality control sorter.*

This worker reads the clipboard to find the raw data, filters out the junk (non-PLA materials), fixes missing values, saves a pristine clean dataset, and updates the clipboard with the new clean file path.

Stage 03 — The Surrogate Model Worker (`stage_03_surrogate.py`). *Job: The mathematical engine.*

This worker reads the clean data path and trains a Gaussian Process Regressor (GPR) to understand the physics of the 3D printer, and writes the model's accuracy score (R^2) and uncertainty metrics onto the clipboard.

Stage 04 — The Acquisition Worker (`stage_04_acquisition.py`). *Job: The explorer.*

This worker looks at the model's uncertainty on the clipboard and hunts for the 100 most unknown configurations (blind spots). It generates these new synthetic rows, wraps them into a lightweight JSON string, stores that string to the board, and sends it back to Worker 3 to retrain. *(This cycle repeats until 10,000 rows are made.)*

Stage 05 — The Validation Worker (`stage_05_validation.py`). *Job: The final checker.*

Once the loop is completed, this worker reads the final synthetic dataset and checks it against the strict laws of thermodynamics and hardware limits. If it passes, they stamp “Validation: Passed” on the clipboard and the pipeline safely shuts down.

SYSTEM ARCHITECTURE: THE STATE TELEMETRY REGISTRY (`State.py`)

What Is the Role of This File?

The `state.py` acts as a **clipboard that travels down the conveyor belt**. It helps to let the other workers later on the line know what work is done and enforces a rigid, immutable contract for that clipboard. It guarantees that every node exactly knows what data exists, its name, and what datatype it is.

Problem. Usually when Python scripts are written the Global Variables are used. We define `data = []` at the top of the file, and every function edits that same list. But in a **Directed Acyclic Graph (DAG)** like LangGraph, global variables are dangerous. Our Bayesian Active Learning (BAL) loop is going to run hundreds of times, going back and forth between the GPR (Stage 03) and the Optimizer (Stage 04). If they both try to read and write to a global variable at the same time, it causes a “Race Condition” and the system crashes.

Solution. To overcome the above problem we built `state.py` which acts as an **immutable registry**. It defines a strict set of rules for a dictionary. No node is allowed to make changes in the dictionary directly. Instead, a node reads the dictionary, does its work, and hands back a small *update sheet*. The pipeline controller then safely merges that update sheet into the master dictionary before passing it down to the next node. So `state.py` acts as a centralized shared memory structure that stores both pipeline data and runtime diagnostic information for all execution stages — the master shared state object that tracks everything happening in the pipeline.

Code Overview and Explanation

The Contract: TypedDict

```
1 class DigitalTwinState(TypedDict, total=False):
```

Code Listing 1: TypedDict class definition for the pipeline state schema.

`state.py` is a `TypedDict`. In Python, a `TypedDict` does not exist at runtime. When we run `python main.py`, the Python interpreter actually ignores `state.py` completely. `state.py` is the **Blueprint** that tells: “If a dictionary is built, it must look like this.”

Standard Python dictionaries (`{}`) are chaotic. We can put anything in them; so to avoid that we use `TypedDict`, which enforces Python to treat this dictionary like a C++ or Java struct. In this way, if a later node tries to read `state["gpr_score"]` instead of `state["r2_score"]`, the system catches it *before the code even runs*. `TypedDict` acts as a spell-checker for our pipeline. If a node tries to create a new variable name, the Python compiler throws an error before the code even runs.

Why use `(total=False)`: This is the most important line in the file. Basically, when LangGraph starts at Stage 01, the state is mostly empty. Stage 05 hasn't run yet, so `validation_passed` doesn't exist. If we didn't include `total=False`, Python's strict typing would instantly crash the program on launch, as we would be missing 15 required variables.

Making `(total=False)` tells the compiler to allow missing keys as the dictionary is built incrementally, but enforce the types for the keys which are already present.

The Pointer-Based Memory Architecture

```
1 raw_data_path: Optional[str]
2 cleaned_data_path: Optional[str]
3 surrogate_model_path: Optional[str]
```

Code Listing 2: Pointer-based file path fields in the pipeline state.

The Problem. A major flaw in ML pipelines is **Memory Overflow**. Such a large ML pipeline will hold entire massive Pandas DataFrames in memory. Passing a 10,000-row DataFrame between 5 different nodes 100 times in a loop will cause the computer's RAM to max out and crash.

The Solution. To prevent a RAM crash we are actually passing `raw_data_path` (a *string*) in the state instead of passing `raw_data` (the actual data). Basically we are following the **Pointer-Based State** pattern. The nodes store the heavy CSV files to the hard drive, and the state only carries the *address* of them.

The Active Learning Loop Breakers

```

1 bal_iteration:          int
2 bal_rows_generated:    int
3 bal_accumulated_data:  Optional[str]    # Serialized JSON matrix
4 synthetic_data_path:   Optional[str]    # Path to final BAL
                             generated CSV
5 synthetic_row_count:   Optional[str]    # Total volume of
                             synthetic rows

```

Code Listing 3: BAL loop tracking variables in the pipeline state.

What it does. This is the tracking mechanism for our BAL loop.

Why it is needed. Basically it tracks the BAL loop. The loop exit condition is implemented through this; otherwise it would become infinite if no exit condition were present.

Stage 04 updates `bal_rows_generated` (e.g., changing it from 300 to 400). When the orchestrator reads the state, it sees 400. It checks the config file (which says 10,000). Because $400 < 10,000$, it loops back to Stage 03. Once that number hits 10,000, the loop breaks and routes to Stage 05.

Why `bal_accumulated_data` is a string. We serialize the new synthetic data into JSON format before putting it in the state. JSON strings are extremely lightweight and fast to transfer between nodes compared to heavy Python objects. Instead of passing the next node the heavy Pandas object, Stage 04 translates just the *raw text and numbers* into a JSON string.

```

1 '{"nozzle_temperature": {"0": 210.5, "1": 215.0},
2   "print_speed":         {"0": 55.2, "1": 60.0},
3   "layer_height":        {"0": 0.15, "1": 0.20}}'

```

Code Listing 4: Example JSON-serialized synthetic data batch passed via state.

Important Clarifications

1. Why did you use LangGraph's state passing instead of just putting the data in an SQLite database?

Database reads and writes are computationally expensive (I/O bound). Because our Bayesian Active Learning loop requires thousands of fast, cyclical micro-transactions between the Surrogate Model and the Optimizer, a disk-based database would bottleneck the system. The `TypedDict` state keeps the orchestration telemetry entirely in high-speed memory while pushing the heavy payloads (the CSVs) to disk, giving us the perfect balance of speed and stability.

2. Is `DigitalTwinState` a global object that all nodes share simultaneously?

No. It utilizes **Immutable State Passing**. `DigitalTwinState` is just a schema. At runtime, LangGraph passes a dictionary to a node. The node returns a partial update (a “diff”). The LangGraph execution engine securely merges that diff into the master state before passing it to the next node. This eliminates race conditions entirely.

3. What actually *is* the data?

In Stage 04 (the Optimizer), the math engine hunts down the 100 most uncertain points in the 3D printer’s physical design space. When it finds them, it creates a small table (a Pandas DataFrame) of 100 new synthetic 3D printer configurations.

4. Why don’t we just put that table directly into the State?

A Pandas DataFrame is a **heavy Python object**. It is not just the numbers; it carries a huge amount of hidden background code (indexes, data type mappings, memory allocation rules, and hundreds of built-in math functions).

If we pass a heavy Pandas object through a LangGraph state loop 100 times, Python’s memory manager has to constantly package and unpackage that heavy object. RAM usage will spike, and the program will drastically slow down or might crash also.

5. The Lifecycle of the Data.

Here is the exact journey of those generated rows:

1. **Stage 04 (Optimizer):** Generates 100 new rows as a Pandas DataFrame.
2. **The Translation:** Stage 04 converts that DataFrame into the JSON string.
3. **The State Update:** Stage 04 writes that lightweight JSON string onto the clipboard (`state["bal_accumulated_data"] = json_string`).
4. **The Graph Hops:** The Orchestrator takes the clipboard and passes it safely to Stage 03.
5. **Stage 03 (Surrogate):** Reads the JSON string off the clipboard (Python Dictionary), uses `pd.read_json()` to instantly convert it back into a usable Pandas DataFrame, and adds those 100 rows to the main CSV file on the hard drive to retrain the model.

SYSTEM ARCHITECTURE: RECEIVING DOCK

(`stage_01_ingestion.py`)

What Does This Node Do?

Stage 01 is the **Receiving station** of our factory. Its main job is to go to the external world (which in our case is Kaggle), get the raw 3D printer data, bring it inside, and write its location on the clipboard (`state.py` — dictionary). This also makes our pipeline modular and serves to define **loose coupling**, i.e., preventing the hardcoded links and single data source dependency.

What Is the Need for This Node?

Instead of directly downloading the data from Kaggle through a dataframe as:

```
1 df = pd.read_csv("https://kaggle.com/...")
```

Code Listing 5: Naive direct download approach — not used in the pipeline.

Why are we creating a separate dedicated node for this task?

The Problem. For an ML pipeline, the above approach has three main catastrophic design flaws:

- **Network Fragility:** If the internet drops for even one second, the whole pipeline crashes.
- **API Rate Limiting:** If our loop runs 100 times and tries to download the file 100 times, Kaggle will ban our IP address. (Rate limiting was implemented to bypass the Kaggle API limits during the development phase.)
- **Memory Bloat:** Downloading directly into Pandas stores the data in RAM, which we specifically avoided when designing `state.py`.

The Solution. To tackle the above problems and make our pipeline modular and source independent, we created an **Idempotent Ingestion Engine**. “Idempotent” here simply means: *It does not matter how many times we run this node, the result is exactly the same, there are no unnecessary repeated API calls for the same purpose, and it does not waste resources*, i.e., if the dataset file isn’t there in the system, it downloads it; if the file is already present, it silently skips the download.

Code Overview and Explanation

The Custom Exceptions (Fail-Fast Methodology)

```
1 class KaggleAuthError(Exception):  
2     pass  
3  
4 class DatasetRetrievalError(Exception):  
5     pass
```

Code Listing 6: Custom exception classes for fail-fast error handling in Stage 01.

The Problem. When an internet download fails, Python throws a generic `ConnectionError` and it becomes difficult to identify the main cause of the error: whether the network connection is down, the server or data source (Kaggle) is down, or the password is wrong.

The Solution. To handle such errors, exception handling is done by making custom exceptions such as `DatasetRetrievalError` and `KaggleAuthError`.

Code. If the system doesn't have Kaggle credentials, we manually trigger `KaggleAuthError`. This is called the “**Fail-Fast**” principle. Instead of trying to connect and waiting 60 seconds for a timeout, the code instantly stops and prompts the user with the error message "Missing API credentials in environment payload.".

The Engine Overview (Encapsulation)

```
1 class DataIngestionEngine:
2     def __init__(self, target_directory: str,
3                 filename: str, remote_slug: str):
4         self.storage_dir = Path(target_directory)
5         self.target_file = self.storage_dir / filename
6         self.repository_slug = remote_slug
```

Code Listing 7: `DataIngestionEngine` class initialization with cross-platform `Pathlib` support.

The Problem. Hardcoding file paths like "C:/Users/himan/..." directly into the downloading logic would require extensive setup on every new system and altering these hardcoded paths on every new machine; otherwise the code breaks.

The Solution. To tackle the above problem we use **Object-Oriented Encapsulation** combined with Python's `Pathlib`.

Code. `__init__` sets up the engine. It takes the target folder, the filename, and the Kaggle link (`remote_slug`) as input variables. `Pathlib` automatically formats the slashes (/) correctly irrespective of the machine type, whether the pipeline is run on Windows, Mac, or Linux.

The Internal Checks (Private Methods)

```
1 def _verify_environment(self) -> None:
2     # Checks for KAGGLE_USERNAME and KAGGLE_KEY in the system
3
4 def _pull_from_remote(self) -> None:
5     # Actually triggers the Kaggle API download
6
7 def _validate_integrity(self) -> None:
8     # Opens the downloaded CSV and checks if it has 0 rows
```

Code Listing 8: Private method stubs for environment verification, remote pull, and integrity validation.

The Problem. Downloading a file has three different associated risks:

- *No permission* (permission to access the file or server is denied)
- *Network failure* (connection not established)
- *Corrupted data* (the data gets corrupted, making it of no use)

The Solution. We handle the above problems separately through private methods, each for a specific purpose, since trying to do it all at once in a single function makes the code too messy and very hard to debug.

The Smart Caching

```

1 def execute_pipeline(self) -> Path:
2     self.storage_dir.mkdir(parents=True, exist_ok=True)
3
4     if self.target_file.exists():
5         logger.info("Local artifact detected. Bypassing
6         network call.")
7     else:
8         self._verify_environment()
9         self._pull_from_remote()
10        self._validate_integrity()
11
12    return self.target_file

```

Code Listing 9: Idempotent `execute_pipeline` method with local artifact caching.

The Problem. The API Rate Limiting issue we mentioned earlier. We don't want to download a raw dataset file every single time we test the code.

The Solution. The approach we followed here is *download once and use infinitely*. So we implemented **Local Artifact Caching** which basically downloads and stores the file and, the next time it is needed, checks whether it already exists and then bypasses the download.

Code. `self.target_file.exists()` checks the hard drive first. If `ultimaker_s5.csv` is already present in the `data/raw/` folder, it completely skips the network call. If it isn't there, it runs the three private methods defined above. Either way, it returns the exact file path to the next step.

The LangGraph Translator (The Node Interface)

```

1 def execute_ingestion(state: DigitalTwinState) ->
    DigitalTwinState:

```

```

2      # ... logging ...
3      engine = DataIngestionEngine(
4          target_directory = "data/raw",
5          filename         = "ultimaker_s5.csv",
6          remote_slug      = "afumetto/3dprinter"
7      )
8
9      final_path = engine.execute_pipeline()
10
11     return {
12         "current_node":      "stage_01_ingestion",
13         "raw_data_path":     str(final_path.resolve()),
14         "pipeline_status":   "running"
15     }

```

Code Listing 10: LangGraph node interface function for Stage 01 ingestion.

The Problem. LangGraph (the Boss) is very rigid. It only knows how to pass the `state.py` clipboard. It does not know what Kaggle is, and how to run a `DataIngestionEngine`.

On the other hand, the `DataIngestionEngine` (the Machine) is highly specialized. It only knows how to download files. It is not aware of the context, what a LangGraph clipboard is.

The Solution. To get the work done we need to establish the bridge between the two. The `execute_ingestion` function is the **Operator** or **Translator** between the (LangGraph) Boss and the (DataIngestionEngine) Machine.

Here is exactly what the function does:

- **Takes the Clipboard:** It accepts `state: DigitalTwinState` (which is the actual, physical **clipboard**, i.e., the Python dictionary) from the Boss.
- **Builds the Machine:** It creates the `DataIngestionEngine` with the exact configurations it requires:

```

1  engine = DataIngestionEngine(
2      target_directory = "data/raw",
3      filename         = "ultimaker_s5.csv",
4      remote_slug      = "afumetto/3dprinter"
5  )

```

Code Listing 11: Creating `DataIngestionEngine` with dataset-specific configuration.

- **Starts the Pipeline:** Then it tells the engine to execute the pipeline by calling the function `execute_pipeline()` and waits for the engine to complete execution and hand back the file path (`final_path`).
- **Translates the Result:** A Python Path object is returned from the engine; a

dictionary update is required for LangGraph. This function takes the output and translates it into the exact format required by the contract defined within `state.py` and returns the dictionary:

```
1 return {  
2     "current_node":      "stage_01_ingest",  
3     "raw_data_path":     str(final_path.resolve()),  
4     "raw_row_count":     engine.extracted_row_count,  
5     "error_message":     None,  
6     "pipeline_status":   "running"  
7 }
```

Code Listing 12: State update dictionary returned from `execute_ingestion`.

Code. We start with the state clipboard as input to this function. Then create an instance of the Engine and use it to retrieve the data, and return the `final_path` as the result from the engine instance. We also create a small dictionary update sheet containing the same keys defined in `state.py` (`raw_data_path`), and return this to LangGraph for the purpose of being merged with LangGraph.

If the pipeline encounters a failure, the system instantly triggers an exception. This is then captured to ensure the relevant error message is recorded on the clipboard (`error_message`) and the terminal logs are updated for diagnostic tracking.

Important Clarifications

1. Why did you write a 120+ line class just to download a CSV file? Is that over-engineering?

It might look like over-engineering if we look from the lens of a standard script. *But in an autonomous Digital Twin, data ingestion must be deterministic and fault-tolerant.*

By encapsulating the Kaggle API in an Idempotent Ingestion Engine, we separated the network logic from the mathematical pipeline. The engine implements ***strict “Fail-Fast” environment checks, physical integrity validation (to prevent corrupt zero-byte files from messing the surrogate model), and local artifact caching.***

This caching ensures that even if the LangGraph orchestrator restarts, it doesn't execute redundant network calls and thus protects the system against API rate limits and minimizes cloud latency.

2. Why didn't you just put the download code directly inside the LangGraph node function rather than create a separate Engine class?

We followed the principle of **Separation of Concerns (Decoupling)**.

The LangGraph node function is only responsible for state management and routing.

The actual downloading of data is business logic, which belongs to an isolated class. By keeping them separate, we can easily remove the Kaggle engine and replace it with an AWS S3 engine or any other external resource in the future without ever having to touch or rewrite the LangGraph orchestration logic.

3. If we make everything modular then why do we hardcode the dataset parameters while passing to DataIngestionEngine?

```
1 engine = DataIngestionEngine(  
2     target_directory = "data/raw",  
3     # Specifies where to store the downloaded dataset.  
4     # Keeping raw  
5     # data in a separate folder helps keep the project  
6     # organized.  
7     filename          = "ultimaker_s5.csv",  
8     # Defines the exact name of the saved file. Later stages  
9     # of the  
10    # pipeline can easily find and use it without relying on  
11    # inconsistent default download names.  
12    remote_slug        = "afumetto/3dprinter"  
13    # Serves as the dataset's unique identifier on Kaggle.  
14    # The API  
15    # uses this slug to find and download the correct dataset  
16    # from  
17    # Kaggle's servers.  
18 )
```

Code Listing 13: Configurable DataIngestionEngine instantiation with annotated parameters.

We did so to make the DataIngestionEngine reusable. If later in the future we decide to build a completely separate Digital Twin for an *Ender 3* printer using a different dataset, and if we hardcoded "afumetto/3dprinter" deep inside the Engine's internal code, we would have to rip the Engine apart and rewrite it.

Because we designed the Engine to accept these configurations from the outside, the Engine itself is just a "blind machine." It just follows orders. To build an Ender 3 twin, we don't have to rewrite the Engine at all from scratch; we just need to change the settings and pass them in:

```
1 # Reusing the exact same engine for a totally different  
2 # project  
3 engine = DataIngestionEngine(  
4     target_directory = "data/raw",  
5     filename          = "ender_3.csv",  
6     remote_slug        = "some_other_creator/ender_dataset"  
7 )
```

Code Listing 14: Reusing DataIngestionEngine for a different dataset project without modifying internals.

By passing the settings at the LangGraph node level, the underlying Machine remains perfectly reusable for any future Machine Learning project we ever build.

Coding Implementation Notes

```
1 def execute_ingestion(state: DigitalTwinState) ->
    DigitalTwinState:
```

Code Listing 15: LangGraph node function signature with type hints for Stage 01.

1. state (The Physical Clipboard). The word **state** on the left side of the colon is just a variable name. It is the actual, physical **clipboard** (the Python dictionary) that the Boss (LangGraph) is handing to Worker 1 right at this exact millisecond.

If we print the state right as the function started, it would just look like a dictionary: `{"retry_count": 0, "pipeline_status": "running"}`.

2. : DigitalTwinState (The Security Badge). The right side of the colon is a Python feature called a **Type Hint**.

We built `DigitalTwinState` in `state.py` to act as the blueprint/rules for the clipboard. By writing `: DigitalTwinState`, we are attaching a security badge to this function.

It tells Python, the IDE (like VS Code), and the LangGraph engine:

*"I am accepting a variable named **state**. But do not hand me just any random dictionary. The dictionary you hand me **MUST** perfectly match the rules defined in **DigitalTwinState**."*

3. -> DigitalTwinState (The Return Promise). The arrow at the end is the worker's promise to the Boss. It states that *when the data is downloaded, the update sheet which this function will return will **ALSO** perfectly follow the rules of the **DigitalTwinState** blueprint.*

4. The Security Mechanism (`_verify_environment`).

```
1 def _verify_environment(self) -> None:
2     """Validates the presence of required API keys."""
3     if not os.environ.get("KAGGLE_USERNAME") or \
4         not os.environ.get("KAGGLE_KEY"):
5         raise KaggleAuthError(
6             "Missing API credentials in environment payload."
7         )
```

Code Listing 16: Environment credential verification to protect API keys from exposure.

If we write Kaggle username and password directly into `stage_01_ingestion.py`, and then upload the project to GitHub or deploy it, bots will steal those credentials in seconds. Currently our source is Kaggle, but later when the source or data is changed, we must protect the credentials to access that data.

The `_verify_environment` function acts as a **Security Checkpoint**. It doesn't look at the code for credentials; it looks at the computer's hidden OS files (specifically, a file called `~/.kaggle/kaggle.json`). If that file exists, Kaggle's API silently and securely lets the program through. If it doesn't, our `KaggleAuthError` immediately stops the program to prevent a crash.

SYSTEM ARCHITECTURE: QUALITY CONTROL SORTER (stage_02_cleaning.py)

What Does This Node Do?

Its main job is to take the raw materials (the messy Kaggle dataset) from the hard drive, filter out the junk (non-PLA materials), fill in any missing gaps (imputation), and ensure the dataset is mathematically sound before passing it to the mathematical engine (Stage 03).

What Is the Need for This Node?

Instead of just passing the raw data directly to the Gaussian Process Regressor, we created a dedicated sanitation node.

The Problem. Raw datasets contain three catastrophic flaws for a Machine Learning pipeline:

- **Schema Chaos:** Column names have spaces, random capitalizations, and typos.
- **Missing Values (NaN):** A single missing number will cause the gradient descent calculus in the Surrogate Model to explode, crashing the entire pipeline.
- **Irrelevant Noise:** The dataset contains prints made of ABS or PETG. Training a digital twin of a PLA print process using ABS data will severely poison the model's physics engine.

The Solution. To tackle the above problems and make our pipeline robust, we built the `DataSanitizer` class. This acts as a strict physical filter. It normalizes the data, drops the noise, mathematically imputes the missing gaps, and acts as a "Gatekeeper" that will deliberately crash the pipeline if the resulting data isn't large enough to train a model.

Code Overview and Explanation

The Custom Exceptions (Fail-Fast Methodology)

```
1 class DataIngestionError(Exception):
2     pass
3
4 class InsufficientDataError(Exception):
5     pass
```

Code Listing 17: Custom exception classes for fail-fast error handling in Stage 02.

The Problem. If data is missing or too small, standard pandas will just output an empty DataFrame, causing bizarre, hard-to-trace math errors deep inside the Stage 03 GPR training loop.

The Solution. We implement the “Fail-Fast” principle. `DataIngestionError` triggers if the file doesn’t exist. `InsufficientDataError` triggers if our cleaning process leaves us with too little data to train a model. Instead of waiting for the math to break, the pipeline instantly stops and reports exactly what went wrong.

The Engine Overview (Encapsulation)

```
1 class DataSanitizer:
2     def __init__(self, raw_filepath: str):
3         self.raw_path = Path(raw_filepath)
4         self.target_material = "pla"
5         self.min_viability_threshold = 15
```

Code Listing 18: `DataSanitizer` class initialization with material target and viability threshold.

Code. `__init__` sets up the engine. It defines our absolute rules: we are targeting "pla", and we refuse to proceed if we have fewer than 15 valid rows. By encapsulating this in a class, we avoid floating procedural code and prevent global variable leaks.

Schema Normalization

```
1 def load_and_standardize(self) -> None:
2     normalized_cols = [
3         str(col).strip().lower().replace(" ", "_")
4         for col in self._dataframe.columns
5     ]
```

Code Listing 19: Column name normalization to enforce a rigid programmatic schema.

The Problem. Human-entered column names like "Print Speed " or "layer-Height" cause dictionary key errors later in the pipeline.

The Solution. We strip hidden whitespace, convert everything to lowercase, and replace spaces with underscores. "Print Speed " becomes `print_speed`. This creates

a rigid, programmatic schema.

Target Isolation (Boolean Masking)

```

1 def isolate_target_material(self) -> None:
2     material_mask = (
3         self._dataframe[self.material_column].str.strip()
4         .str.lower()
5         == self.target_material
6     )
7     self._dataframe = self._dataframe[material_mask].copy()

```

Code Listing 20: Boolean mask to isolate PLA rows and discard ABS and PETG data.

Code. This is where we filter out ABS and PETG. We create a `material_mask` (a True/False checklist). The DataFrame overlays this mask and instantly drops any row that isn't True (PLA). The `.copy()` ensures we detach from the old memory block, preventing Pandas "SettingWithCopy" warnings.

Anomaly Mitigation (Mathematical Imputation)

```

1 def mitigate_anomalies(self) -> None:
2     if not num_schema.empty:
3         medians = self._dataframe[num_schema].median()
4         self._dataframe[num_schema] = (
5             self._dataframe[num_schema].fillna(medians)
6         )

```

Code Listing 21: Median and mode imputation to repair missing values without row deletion.

Code. Machine learning models cannot process null (missing) values. If a temperature reading is missing, we use `fillna(medians)` to inject the exact middle value of that specific column. For text columns, we inject the mode (the most common value). This structurally repairs the data without needing to delete valuable rows.

The LangGraph Translator (The Node Interface)

```

1 def execute_cleaning(state: DigitalTwinState) ->
   DigitalTwinState:
2
3     sanitizer = DataSanitizer(raw_path)
4     sanitizer.load_and_standardize()
5     sanitizer.isolate_target_material()
6     sanitizer.mitigate_anomalies()
7     sanitizer.validate_viability()
8
9     output_loc = sanitizer.export_payload(
10         Path("data/interim"), "pla_cleaned.csv"
11     )
12
13     return {
14         "current_node": "stage_02_clean",

```

```
15         "cleaned_data_path": str(output_loc.resolve()),
16         "eda_report":        profile,
17         "pipeline_status":    "running"
18     }
```

Code Listing 22: LangGraph node interface function for Stage 02 cleaning.

The Problem. LangGraph (the Boss) only understands the `state.py` clipboard. It doesn't know how to clean data. The `DataSanitizer` (the Machine) only knows how to clean data, but doesn't understand LangGraph.

The Solution. The `execute_cleaning` function is the **Translator**.

- **Reads the Clipboard:** It reads the raw data path from the `state.py` clipboard.
- **Boots the Machine:** It starts up the `DataSanitizer` and runs the exact cleaning sequence.
- **Saves the Result:** It saves the pristine `pla_cleaned.csv` to the hard drive.
- **Translates Back:** It translates the results into a dictionary update sheet, giving LangGraph the new `cleaned_data_path` and an `eda_report` (Exploratory Data Analysis) to pass down the line.

Important Clarifications

1. Why did we use Median/Mode imputation instead of just dropping rows with missing data or using the Mean (Average)?

Because we operate in a “Small Data” regime. In 3D printing datasets, every physical test point is highly expensive. If we drop rows with missing data, we might lose 30% of our dataset, triggering our `InsufficientDataError`.

Furthermore, we use the Median instead of the Mean because the Mean is easily skewed by extreme outliers (e.g., a sensor glitch recording 5000°C). The Median mathematically guarantees we are injecting a physically realistic, middle-of-the-road value that won't poison the Gaussian Process Regressor in Stage 03.

2. Why did you write an entire class (`DataSanitizer`) just to clean a pandas `DataFrame`? Isn't that over-engineering?

In a standard Jupyter Notebook, this would be over-engineering. But in an autonomous Digital Twin, data flow must be strictly encapsulated. By wrapping the logic in a class, we track internal state (like `self._initial_row_count` vs. `self.final_row_count`) safely. It completely separates the business logic of data cleaning from the orchestration logic of LangGraph, allowing us to swap rules in the future without breaking the DAG pipeline.

3. What is the `generate_statistical_profile` (EDA Report) used for?

When the data is cleaned, the Sanitizer generates a small dictionary containing the dataset's shape, remaining nulls, and numeric summaries. This is attached to the Lang-Graph clipboard. It acts as runtime telemetry — if the pipeline crashes in Stage 04, developers can look at the Stage 02 EDA Report in the state history to instantly see what the data looked like the moment it left the Quality Control Sorter.

SYSTEM ARCHITECTURE: BLINDFOLDED SURVEYOR (`stage_03_surrogate.py`)

The Concept: What Is a Gaussian Process Regressor?

Let us understand with the help of a simple example.

Suppose we are a surveyor in a vast, hilly field, but we are completely blindfolded and our goal is to draw a 3D map of the entire area.

We take a step, reach down, and measure the ground where we find that we are exactly 10 feet above sea level. Then we walk 20 paces to the right, measure again, and discover that we are 12 feet above sea level.

Now here comes the understanding and *the logic why we choose GPR*.

If we give this data to a Standard Machine Learning model (like a Neural Network) which looks at our two footprints and boldly guesses the ground everywhere else, it pretends to know everything, even in areas where it hasn't stepped.

The Gaussian Process Regressor (The Smart Guesser) does something clever. It guesses the height and also tells you how certain it is about its guess.

If we ask the GPR about the ground 1 inch away from our footprint, it will say: “*I predict it is 10.1 feet high, and I am 99% confident.*” If we ask it about a spot a mile away from where we haven't measured yet, it will say: “*I predict it is 11 feet high, but my uncertainty (σ) is huge. It could be a mountain, or it could be a hole. I don't know.*”

Why Is This Required for the 3D Printer?

Printing 3D parts takes hours, and testing them destroys them, which in all is an expensive task both financially and in time. We currently only have a few physical data points (the footprints). We use the GPR to mathematically guess the physics of the printer for the other 9,975 points, and its “uncertainty score” tells Stage 04 (The Hunter) exactly where it needs to explore next to improve the map.

1. **The Field, or Design Space.** Think of the rolling hills here as the hard physical reality of the Ultimaker S5. There is a mathematical truth to how Nozzle Temperature and Tensile Strength are related; we are after that, yet we can never view it in its entirety.
2. **A Blindfold.** To the AI, there is no prior knowledge. It has no concept of thermodynamics or the fact that plastics have a melting point. As far as it is concerned, the way the physical world operates is something it is entirely blind to.
3. **Footprints from your data.** This is where your cleaned data comes in. Stage 02 hands the GPR a CSV with 25 rows of data, and those become the footprints. The AI will lift its blindfold for a millisecond to examine them and make note of the specifics:

“Okay, at exactly 210°C, the strength is exactly 45 MPa.”

“At exactly 215°C, it is 48 MPa.”

4. **The Map.** Now the GPR fills the blanks. Having seen 210 and 215, it will put a line between them and surmise that 212 is in the vicinity of 46.5 MPa. That is the mean prediction, the model you will find on your hard drive.
5. **The Uncertainty Bounds (σ).** Here is the power of GPR. Put a standard Neural Network in front of 250°C, a temperature it has not encountered, and it will lie to you with a confident but random number. GPR is more forthright. Being a probabilistic model, it knows what it does not know. You ask it about 250°C and it will tell you its best guess is 30 MPa, but with an uncertainty of ± 50 because there are no footprints in that area to guide it. In short, it has no idea. Stage 04, the Hunter, is designed to spot these uncertainties and tell us where to run our next simulation.

Understanding GPR: Gaussian Process Regression

Non-Parametric Machine Learning Model

A non-parametric machine learning model is a model that does not make strong assumptions about the form of the functions that are being learned. Predictions for a new data point are made based on how similar the features of the new point are to data points in the training set in the feature space.

Some examples are: Decision trees, kernel-based SVM, k-nearest neighbour (KNN), and Gaussian Process Regression (GPR), as opposed to parametric models such as linear regression, ridge regression, LASSO, and neural networks.

Gaussian Process Regression (GPR)

GPR, also known as **kriging**, is a Bayesian approach to machine learning. In this approach, the prior distribution of the space of functions is assumed to be a Gaussian

Process and the observed data is used to determine the posterior distribution of the function being approximated, from which the target variable is predicted.

Mathematics Behind GPR. The prior assumption is that the function we are approximating is a Gaussian process, which is characterized by a mean and a covariance matrix. The mean functional is essentially the expected value of the function we are trying to approximate.

A Gaussian Process is defined as:

$$f(x) \sim \mathcal{GP}(m(x), k(x, x')) \quad (1)$$

where $m(x)$ is the expected value of the function and $k(x, x')$ computes the covariance matrix (kernel function), which measures how strongly the outputs at x and x' are related:

$$m(x) = \mathbb{E}[f(x)] \quad (2)$$

$$k(x, x') = \mathbb{E}[(f(x) - m(x))(f(x') - m(x')))] \quad (3)$$

The kernel function $k(x, x')$ of the GPR is given as:

$$k(x, x') = \sigma^2 \exp\left(-\frac{\|x - x'\|^2}{2\ell^2}\right) \quad (4)$$

where:

- σ^2 = variance parameter
- ℓ = length scale
- $\|x - x'\|$ = distance between points

This simply interprets that nearby points have high similarity and far points have low similarity.

A Gaussian Process (GP) is a stochastic process characterized by a collection of random variables such that every finite collection of those random variables follows a multivariate normal distribution.

Gaussian (Normal) Distribution

$$X \sim \mathcal{N}(\mu, \sigma^2) \quad (5)$$

Meaning:

- μ = mean (average value)
- σ^2 = variance (spread of data)
- σ = standard deviation

The fundamental statistical equations are:

$$\mu = \mathbb{E}[X] \quad (\text{Mean Equation}) \quad (6)$$

$$\sigma^2 = \mathbb{E}[(X - \mu)^2] \quad (\text{Variance Equation}) \quad (7)$$

$$\sigma = \sqrt{\sigma^2} \quad (\text{Standard Deviation}) \quad (8)$$

Example. $X \sim \mathcal{N}(5, 1)$:

- Mean = 5
- Variance = 1
- Standard deviation = 1
- Most values lie close to the mean.
- Farther values are less probable.

Joint Probability Density Function (Multivariate Gaussian in GP)

In the case of a GP, the collection of random variables follow the joint probability distribution function based on the features, which is given as:

$$p(f | X) = \frac{1}{\sqrt{(2\pi)^n |K|}} \exp\left(-\frac{1}{2} (f - m)^\top K^{-1} (f - m)\right) \quad (9)$$

where:

- f = vector of function values
- m = mean vector
- K = covariance matrix
- $|K|$ = determinant of covariance matrix
- K^{-1} = inverse of covariance matrix
- n = number of data points or random variables
- $\exp(\cdot)$ = exponential function

One important point which needs to be noted here is the **Condition on Covariance Matrix**:

$$|K| > 0 \quad (10)$$

This ensures that:

1. K is invertible.
2. The multivariate Gaussian distribution is valid.
3. K is positive definite.

Positive Definite Condition:

$$z^\top K z > 0 \quad \forall z \neq \mathbf{0} \quad (11)$$

In Gaussian Process Regression (GPR), we first assume what the unknown function could roughly look like before seeing any actual training data. This initial assumption is called the **prior distribution**. It is mainly defined using a mean function and a kernel function. The mean function represents the average behavior of the function, while the kernel function tells us how strongly different data points are related to each other.

After observing the training data (X, Y) , this prior belief is updated using Bayesian inference to form the **posterior distribution**. The posterior uses both the prior assumptions and the information learned from the data to make predictions. *Unlike traditional regression models that try to learn only one fixed function, GPR considers many possible functions that could fit the data. It also provides uncertainty estimates along with predictions, which is one of its biggest advantages.*

The Posterior Distribution

Given training data (X, Y) , we assume:

$$y_i = f(x_i) + \varepsilon_i \quad (12)$$

where $\varepsilon_i \sim \mathcal{N}(0, \sigma_n^2)$, meaning ε_i is Gaussian noise with mean = 0 and variance = σ_n^2 .

If the mean of the target variable is assumed to be zero, then:

$$f(x) \sim \mathcal{GP}(0, k(x, x')) \quad (13)$$

where 0 is the zero mean function and $k(x, x')$ is the covariance/kernel function. This means the function values are completely determined by the covariance structure.

Now since the observed outputs contain noise, the target values can be represented as:

$$Y = f + \varepsilon \quad (14)$$

where f = true latent function values, ε = Gaussian noise/error, and Y = actual observed outputs. The noise assumption is:

$$\varepsilon \sim \mathcal{N}(0, \sigma^2 I) \quad (15)$$

Meaning: noise has mean 0, variance = σ^2 , and I means noise is independent for each data point.

Because both the latent function and the noise are Gaussian distributed, the observed outputs also follow a Gaussian distribution:

$$Y \sim \mathcal{N}(0, K_{xx} + \sigma^2 I) \quad (16)$$

Meaning: the covariance of Y now contains (1) function covariance K_{xx} and (2) noise variance $\sigma^2 I$.

We already know training outputs Y . Using those known outputs, we want to predict f^* (function values at unseen test points), for which we calculate f^* given Y :

$$f^* | Y \quad (17)$$

This is called the **Posterior Distribution**. The final Posterior Distribution equation becomes:

$$f^* | Y \sim \mathcal{N}(\mu^*, \Sigma^*) \quad (18)$$

Meaning: Predictions at test points are Gaussian distributed with mean = μ^* and covariance = Σ^* .

Predicted Mean (μ^*). μ^* are the actual predictions made by GPR, whose equation is:

$$\mu^* = K_{x^*x} (K_{xx} + \sigma^2 I)^{-1} Y \quad (19)$$

The intuition is that predictions are obtained using (1) similarity between train and test points, and (2) observed training outputs.

Predicted Covariance (Σ^*). Σ^* is the uncertainty in predictions, whose equation is:

$$\Sigma^* = K_{x^*x^*} - K_{x^*x} (K_{xx} + \sigma^2 I)^{-1} K_{xx^*} \quad (20)$$

This uncertainty comes from the fact that:

- If a test point is close to training data → uncertainty becomes low.
- If it is far from training data → uncertainty becomes high.

So if we have to summarize this whole GPR, we can say that we start with a prior belief about the function, then observe training data, and then update that belief to obtain predictions and uncertainty at new test points. This uncertainty is what makes GPR so powerful and suitable for the task of mapping the gaps in the data, because we never want to produce data which is far from reality, physics, and the actual data.

Code Overview and Explanation

Library Imports and Error Handling

```

1 import logging
2 import pickle
3 import yaml
4 import numpy as np
5 import pandas as pd
6 from pathlib import Path
7 from typing import Dict, Any, Tuple
8
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.gaussian_process import GaussianProcessRegressor
11 from sklearn.gaussian_process.kernels import Matern,
    ConstantKernel
12 from sklearn.model_selection import cross_val_score
13 from agent.state import DigitalTwinState

```

Code Listing 23: Imports and custom exception classes for the surrogate model engine.

- **sklearn:** We utilize Scikit-Learn for statistical modeling and also for using the `GaussianProcessRegressor` in order to use the `gaussian_process` and its kernels.
- **pickle:** This is basically used for object serialization, i.e., saving the trained model to the file system (`.pkl`).
- **SurrogateEngineError:** This is the custom exception class we created to implement the fail-fast principle if the maths fails.

```

1 class SurrogateConvergenceError(Exception):
2     pass
3
4 class MissingDependencyError(Exception):
5     pass

```

Code Listing 24: Custom exception classes for surrogate model failure handling.

Custom Exceptions: Defining custom exceptions to handle pipeline crashes. These logs explicitly state whether it was a math failure (`SurrogateConvergenceError`) or a missing file failure (`MissingDependencyError`).

Building the Mapper (Class Initialization and Hyperparameters)

```

1 class GaussianProcessEngine:
2     def __init__(self, data_filepath: str,
3                   config_filepath: str = "config/
4 printer_params.yaml"):
5         self.data_path = Path(data_filepath)
6         self.config_path = Path(config_filepath)

```

Code Listing 25: GaussianProcessEngine class initialization with cross-platform path handling.

Initialising the Mathematical engine (GaussianProcessEngine), accepting the path to the data and defaulting to our printer_params.yaml configuration file. Path() ensures cross-platform compatibility and makes the structure modular.

```

1 self.training_corpus: pd.DataFrame = pd.DataFrame
2   ()
3 self.feature_cols: list = []
4 self.target_cols: list = []
5 self.scaler = StandardScaler()
6 self.trained_models: Dict[str, GaussianProcessRegressor] = {}
7 self.cv_metrics: Dict[str, float] = {}
8 self.uncertainty_metrics: Dict[str, float] = {}

```

Code Listing 26: Empty placeholder attributes for training data, models, and metrics.

Creating the empty placeholders to store the data produced. The lists and DataFrame will hold the data. The dictionaries at the bottom are where the engine will store the trained models, their R^2 scores, and their uncertainty (σ) scores once training is complete.

Feature Normalization (StandardScaler): Normalization is very important before training the model on the data points. If normalization is not done, features with very large values can dominate features with smaller values, which causes many machine learning algorithms to give poor results.

Dynamic Configuration (_ingest_assets)

```

1 def _ingest_assets(self) -> None:
2     if not self.data_path.exists():
3         raise MissingDependencyError(
4             f"Sanitized payload missing at {self.data_path}"
5         )
6     if not self.config_path.exists():
7         raise MissingDependencyError(

```

```

8         f"Configuration missing at {self.config_path}"
9     )

```

Code Listing 27: Fail-fast file existence checks before loading data and configuration.

Implementing Fail-Fast Checks: Before starting, we verify the existence of CSV and YAML files on the drive using the custom exceptions.

```

1 with open(self.config_path, "r") as cfg_file:
2     config = yaml.safe_load(cfg_file)
3
4 self.feature_cols = list(config["inputs"].keys())
5 self.target_cols = list(config["outputs"].keys())
6 self.training_corpus = pd.read_csv(self.data_path)

```

Code Listing 28: Dynamic schema extraction from YAML configuration file.

Dynamic Schema: We extract keys to define **Features & Targets** from the YAML file, matching the “inputs” block and “outputs” block. This prevents hardcoding the names in the Python script and also gives us the functionality to later modify the inputs and outputs accordingly in the future if the dataset changes by just changing the YAML file. Then we load the actual data into a Pandas DataFrame.

Mathematical Guardrails (`_construct_kernel`)

```

1 def _construct_kernel(self) -> ConstantKernel:
2     return ConstantKernel(1.0, (1e-3, 1e3)) * Matern(
3         length_scale = 1.0,
4         length_scale_bounds = (1e-2, 1e2),
5         nu = 2.5
6     )

```

Code Listing 29: Matern kernel construction with bounded hyperparameters to prevent singular matrices.

Length_scale: It represents the **Correlation Distance**.

Problem. When the Gaussian Process Regressor optimizes its internal map (Maximum Likelihood Estimation), it searches for the mathematically optimal `length_scale`. If we do not give it constraints, the optimizer might attempt to test extreme values (e.g., 10^{15} or 10^{-15}). This causes a **singular matrix error**, leading to the math collapsing into infinity or zero, and the entire pipeline crashes.

The Solution. Therefore we explicitly apply the constraint `length_scale_bounds=(1e-2, 1e2)`: the optimizer can only look at values between 0.01 and 100. This is like a safety net for the math; it helps the training process go smoothly and prevents the pipeline from crashing.

The `length_scale_bounds` do not represent physical hardware limits; they represent hyperparameters for the Matern kernel's correlation distance within the normalized feature space. Bounding the length scale between 10^{-2} and 10^2 ensures the Maximum Likelihood Estimation converges on a physically plausible, continuous function, preventing the mathematical optimizer from collapsing into extreme overfitting (noise) or underfitting (flatline).

`Length_scale_bounds` is a smoothness control parameter. When the model tries to figure out how the printer works, it has to connect the dots. If we let the model control this smoothness however it wants, it usually gets confused and does one of two things:

- **It overthinks:** looks at sensor noise and creates a crazy map with lots of spikes.
- **It gets lazy:** assumes nothing matters; draws a straight line.

By keeping this smoothness between 0.01 and 100 we are giving the model some sense. We are telling the math to draw a map of the printer — don't make it too spiky and don't make it too flat. This way the model gives us a map of how 3D printing materials work.

Core Math and Validation (`execute_training_sequence`)

```
1 def execute_training_sequence(self) -> None:
2     self._ingest_assets()
3     feature_matrix = self.training_corpus[self.feature_cols
4     ].values
5     scaled_features = self.scaler.fit_transform(
6         feature_matrix)
```

Code Listing 30: Feature extraction, normalization, and StandardScaler fit-transform.

Loading the data and isolating the input X values (`feature_matrix`), and then applying `fit_transform` to strictly normalize the inputs to a mean of 0 and variance of 1.

fit_transform: Using the `StandardScaler()` defined above to scale raw X data, storing the result in X_{scaled} .

```
1 for target in self.target_cols:
2     target_vector = self.training_corpus[target].values
```

Code Listing 31: Iterating over each output variable for independent GPR training.

Iterating over each output variable (e.g., Strength, Roughness) one by one.

```
1 gpr = GaussianProcessRegressor(
```

```

2     kernel                = self.kernel,
3     n_restarts_optimizer = 10,
4     alpha                 = 1e-2,    # Noise tolerance
5     random_state          = 42
6 )

```

Code Listing 32: GPR instantiation with multi-start optimization and Tikhonov regularization.

This is the main part of the engine. It loops through 3 targets (Strength, Roughness, Elongation) and trains a separate GPR for each.

n_restarts_optimizer=10: When the GPR tries to fit the map to the initial values, it uses an internal optimizer. Sometimes that optimizer gets stuck in a “local minima” and assumes it found the best map when it actually hasn’t. Initializing **n_restarts_optimizer=10** physically refreshes itself and restarts from a random position 10 times to guarantee it finds the absolute mathematical fit.

random_state=42: This forces the random math to be exactly identical every time we run it, ensuring the pipeline is 100% reproducible for our thesis instead of choosing a new random number every time we run the pipeline and getting different results each time.

The alpha Parameter (Tikhonov Regularization): According to the Statistical Principle, in standard mathematical theory, a GPR assumes the training data to be perfect. If we tell it the temperature is 210.0°C, it believes it is exactly 210.0°C. However, we know that in real-world scenarios sensors may have noise and vibration — in fact they do. By setting **alpha=1e-2**, we are applying ***Tikhonov regularization*** (often known as ridge regression in other contexts). This tells our model to *assume every data point has a $\pm 1\%$ margin of sensor error*.

This stops the model from paying too much attention to tiny changes in the Ultimaker S5 telemetry. This means the model will have a smoother and more general Ultimaker S5 physical map.

```

1 cv_scores = cross_val_score(
2     gpr, scaled_features, target_vector, cv=5, scoring="r2"
3 )
4 mean_cv = float(np.mean(cv_scores))

```

Code Listing 33: 5-fold cross-validation to measure generalization performance and prevent overfitting.

Cross-Validation (cross_val_score, cv=5): Here we basically chop the data into 5 blocks in order to train the model on 4 blocks and test it on the new 1 block. We

do this 5 times; this ensures that the model is actually learning the physics instead of memorizing the data. Before finalizing the model, we evaluate its robustness and prevent overfitting. It returns the R^2 .

The R^2 score (coefficient of determination) is the model's accuracy score for the Gaussian Process Regressor. It acts as a mathematical grade from 0 to 1, where higher values indicate that the model has more accurately learned the underlying physics of the 3D printer.

```
1 gpr.fit(scaled_features, target_vector)
2 train_r2 = gpr.score(scaled_features, target_vector)
```

Code Listing 34: Final GPR fit on full dataset and baseline training accuracy.

It executes the final matrix fit on the entire dataset and calculates the baseline training accuracy.

```
1 _, std_dev = gpr.predict(scaled_features, return_std=True)
2 mean_sigma = float(np.mean(std_dev))
```

Code Listing 35: Predictive variance extraction to quantify model uncertainty (σ).

Predictive Variance (return_std=True): This is the functional core of our Active Learning pipeline. We execute a prediction pass over the known data space and explicitly demand the GPR to return the standard deviation (`std_dev`). It calculates the average σ , which represents the mathematical blind spots of the model.

```
1 self.trained_models[target] = gpr
2 self.cv_metrics[target] = mean_cv
3 self.uncertainty_metrics[target] = mean_sigma
```

Code Listing 36: Storing trained models and metrics into engine dictionaries.

Saving the trained GPR object, the CV score, and the σ score into the dictionaries initialized in `__init__`.

Serialization

```
1 def package_and_export(self, destination_dir: Path,
2                         filename: str) -> Path:
3     destination_dir.mkdir(parents=True, exist_ok=True)
4     export_path = destination_dir / filename
5
6     payload = {
7         "models": self.trained_models,
8         "scaler": self.scaler,
9         "feature_cols": self.feature_cols,
10        "target_cols": self.target_cols
11    }
12
```

```

13     with open(export_path, "wb") as f:
14         pickle.dump(payload, f)
15
16     return export_path
17
18     with open(model_path, "wb") as f:
19         pickle.dump(payload, f)

```

Code Listing 37: Model serialization to disk using pickle for persistent storage.

State Persistence: Since an ML model runs in RAM, it is volatile. The `package_and_export` method bundles all the scaling parameters, feature schemas, and the trained GPR models into a serialized dictionary and stores it to disk (`gpr_package.pkl`). This is standard procedure for model training.

The Problem. Stage 04 (The Hunter) needs to use these trained models. But Stage 03 and Stage 04 do not execute at the same time.

The Solution. We create a payload dictionary which contains the Models, the Scaler, and the column names, and using `pickle.dump()` store this entire package into a single `.pkl` binary file on our drive.

The LangGraph Translator (`fit_surrogate_model`)

```

1 def fit_surrogate_model(state: DigitalTwinState) ->
   DigitalTwinState:
2     try:
3         sanitized_path = state.get("cleaned_data_path")
4
5         training_target_path = sanitized_path
6         if state.get("lhs_data_path") and \
7             Path(state["lhs_data_path"]).exists():
8             training_target_path = state["lhs_data_path"]

```

Code Listing 38: LangGraph node interface for Stage 03 surrogate model fitting.

Just like in Stage 01, this function acts as the **Translator/Operator**. It reads the clipboard (`state.py`) as soon as we get the `cleaned_data_path`, the `SurrogateEngine` is started which trains the GPR on the dataset and exports the `.pkl` file, and then updates the `state.py` clipboard with the exact locations and grades which enables the LangGraph to route it to the next stage (Stage 04, The Hunter).

Here also we have separated the mathematical engine (which contains the core mathematics logic) from the LangGraph (which is responsible for the Orchestration).

```

1 engine = GaussianProcessEngine(
2     data_filepath = training_target_path
3 )
4 engine.execute_training_sequence()

```

```

5
6 overall_cv = float(np.mean(list(engine.cv_metrics.values())))
7
8 package_loc = engine.package_and_export(
9     Path("data/processed"), "gpr_surrogate_model.pkl"
10 )

```

Code Listing 39: Engine instantiation, training execution, and model export for Stage 03.

It instantiates our class, executes the training loop, calculates the aggregate R^2 score across all targets, and exports the .pkl file.

```

1 return {
2     "current_node": "stage_03_fit_gpr",
3     "surrogate_model_path": str(package_loc.resolve()),
4     "surrogate_r2_score": overall_cv,
5     "feature_names": engine.feature_cols,
6     "target_names": engine.target_cols,
7     "gpr_uncertainty_stats": engine.uncertainty_metrics,
8     "error_message": None,
9     "pipeline_status": "running"
10 }

```

Code Listing 40: State update dictionary returned from fit_surrogate_model.

Finally, it creates the dictionary update required by DigitalTwinState, pushing the model path, the accuracy scores, the schemas, and the critical uncertainty metrics back to the global clipboard.

Understanding the Kernel and Its Parameters

1. What Is a Kernel?. *The kernel (or covariance function) in Gaussian Process Regression is like a “rule-book” for the model on how data points relate to each other within the Matern kernel family.*

When we input a data point of temperature at 210°C, the Matern kernel informs the model of how much influence this data point will have at 211°C versus 250°C. This is where the Matern kernel really comes into play.

2. What Is the Matern Kernel?. *In the field of machine learning there are multiple types of kernels. One common type is the RBF (Radial Basis Function) kernel. The RBF kernel makes the assumption that the underlying data is infinitely smooth and has perfect continuity. For example, a sine wave: the underlying data is always smooth.*

The Matern kernel is a family of equations which are advanced and flexible. The Matern kernel does not assume perfect smoothness of the underlying world; it lets us

strictly define just how smooth, or conversely, how “jagged” the model should interpret the physical system. The Matern kernel is especially useful here.

3. What Is ν (Nu)? ν is the mathematical parameter within the Matern kernel that determines the degree of differentiability. In calculus, differentiability is determined by how many times you can compute the derivative (the slope) before the equation itself falls apart.

Here’s how the value of ν affects the model’s interpretation:

- $\nu = 0.5$ (**Not Differentiable**): When we use this value, the model will assume that our data is extremely jagged and chaotic, such as raw electrical static, or the path of stock market fluctuations. It essentially connects the data points with sharply angled zig-zags.
- $\nu = 1.5$ (**Differentiable**): When we set this value, the model assumes that the data is relatively continuous but is able to handle sharp corners.
- $\nu = \infty$ (**Infinitely Differentiable**): Mathematically, this converts the Matern kernel into a standard RBF kernel. The model interprets our data as being flawlessly smooth. For example, the trajectory of a pendulum: this would have infinite differentiability.

4. Why Is It Set Strictly to 2.5? In our FDM Digital Twin setting, setting $\nu = 2.5$ means that the mathematical function has twice differentiability.

Justification:

Physical systems, such as the process of melting PLA on an Ultimaker S5 3D printer, are not perfectly smooth.

If we are using a kernel of infinite smoothness (e.g., the RBF kernel), then the model assumes our printer’s physics is mathematically perfect. Unfortunately, physical processes have definite physical limitations. Plastic has a tensile strength, which means when you try to stretch it past its limit it will break apart (snap), or when heated, it will suddenly melt.

A smooth kernel cannot process the “snap.” The math will then tend to widely overestimate and “hit the roof” of the data when trying to follow the intended line.

Setting $\nu = 2.5$ is the common practice when dealing with physical engineering systems, thermodynamical relationships and spatial mapping applications. This specifically instructs the optimizer to:

“Treat the laws of thermodynamics as being broadly smooth, but allow for

any deviation due to physical limitations to be mapped correctly. For instance, when the plastic is heated enough to melt, or is stressed enough to break, the mathematical function does not fall apart.”

This is why we did not use an RBF kernel for our Gaussian Process, and went for the Matern kernel so it could accommodate the physical limitations of the physical processes.

Important Clarifications

1. Why did we use Gaussian Process Regression instead of a Deep Neural Network even though NNs are much faster?

*Although a standard Neural Network is computationally faster, it gives a deterministic output (a guess) but it cannot mathematically measure its own ignorance. Whereas our Stage 04 Active Learning loop requires an uncertainty metric (**Standard Deviation** / σ) to know where the “blind spots” are in the physical design space. GPR is a probabilistic model that basically calculates this spatial uncertainty, making it the only best choice to drive the Bayesian optimization.*

2. Gaussian Process Regressors have a computational time complexity of $O(N^3)$. A problem may be pointed out here: if we set up a factory of 100 printers and collect 1,000,000 data points, our model may take months to train and our pipeline will crash. Then why are we using a model that can't be scaled?

It looks like a valid concern when we look at Big Data, but the point here is to understand our architecture. Bayesian Active Learning operates in a “Small Data” regime. Physical printing of 3D items takes hours and destroys material; data acquisition is extremely expensive. Our dataset size N will remain small (typically under 1,000 rows). In this regime, the $O(N^3)$ calculation takes milliseconds.

Now the main point is: if we ever reach massive data volumes, the uncertainty σ approaches zero, which means the Active Learning loop is finished, and we can safely swap the GPR for a computationally cheaper, deterministic Deep Neural Network because by that time we will be having the massive accurate dataset needed. So if the problem doesn't exist, why would a solution exist?

3. Why are we looping through targets and training a completely separate GPR model for Tensile Strength, Roughness, and Elongation instead of just training one multi-output model to predict all three at once?

The answer is simply the nature of parameters and their relationships with the input.

We are doing so because these physical properties do not share the same mathematical gradients.

*Surface Roughness is heavily dominated by Layer Height and is mostly unaffected by Nozzle Temperature. Whereas Tensile Strength is the exact opposite: it is dominated by Temperature for layer adhesion. So by training independent GPRs, the optimizer can find a completely different **length_scale** (smoothness) for Roughness than it does for Strength, resulting in a much more highly tuned physics engine. We are trying to get as close as we could get to the real world physics and replicate them in the digital world.*

SYSTEM ARCHITECTURE: QUALITY ASSURANCE GATE (stage_05_validation.py)

This is Stage 05, which is basically the validator, performing the validation before the data goes to the neural network. This is the physical reality check which maintains the physical bounds on the data, thus saving the printer from high or extreme values of the input parameters.

The Machine Learning models (like the GPR or a Neural Network) do not know that PLA melts. For models, all values are just numbers. So let us suppose our Stage 04 generates a synthetic data point which claims that the Nozzle Temperature is -500°C (which violates absolute zero): the math accepts it, but the physical Digital Twin will then try to make the nozzle temperature this low, which is not possible, and neither does PLA melt at this temperature. Stage 05 acts as **Defense in Depth**, not relying on the above nodes to give perfect data every time.

Code Overview and Explanation

Imports and Exception Handling

```

1 import logging
2 import yaml
3 import numpy as np
4 import pandas as pd
5 from pathlib import Path
6 from typing import Dict, Any, List, Tuple
7 from agent.state import DigitalTwinState
8
9 class StateDependencyError(Exception):
10     pass

```

Code Listing 41: Imports and custom exception class for the physics validation engine.

StateDependencyError: We have made a strict failure mode which basically ensures

that if the previous Stage 04 forgets to generate data, our Stage 05 should crash with this specific error rather than proceeding with an empty payload.

Initialization and Physical Constraints (`__init__`)

```

1 class PhysicalConstraintsEngine:
2     def __init__(self, data_path: str,
3                 config_path: str = "config/printer_params.
4                 yaml"):
5         self.target_path = Path(data_path)
6         self.config_path = Path(config_path)
7
8         self.thermodynamic_bounds = {
9             "tension_strength": (0.0, 100.0),
10            "roughness":        (0.0, 500.0),
11            "elongation":        (0.0, 50.0),
12        }

```

Code Listing 42: PhysicalConstraintsEngine initialization with hardcoded thermodynamic output bounds.

The Thermodynamic Ledger: We hardcoded the absolute physical limits of the Ultimaker S5 outputs. For example, if a synthetic data point predicts a Tensile Strength of 150.0 MPa, the engine knows it is physically impossible for PLA to attain so much tensile strength and will flag it.

Metadata Purging (`_initialize_environment`)

```

1 def _initialize_environment(self) -> None:
2     # ... standard load ...
3     raw_df = pd.read_csv(self.target_path)
4     public_schema = [col for col in raw_df.columns
5                     if not col.startswith("_std_")]
6     self._dataframe = raw_df[public_schema]

```

Code Listing 43: Stripping internal GPR uncertainty columns before ANN handoff.

Here we strip off the extra columns which are not needed by our neural network. During Stage 04, to track the GPR's uncertainty, the pipeline appends a column `_std_tension`. However, the final Deep Neural Network (ANN) only requires the inputs and the outputs for training, not the GPR's mathematical metadata. So we strip this column from the final data, thus preventing confusion at our neural network with the unnecessary metadata.

Structural Integrity (`_evaluate_structural_integrity`)

```

1 def _evaluate_structural_integrity(self) -> None:
2     # ... volume check ...
3     null_count = int(self._dataframe.isnull().sum().sum())
4     inf_count  = int(np.isinf(num_matrix).sum())

```

Code Listing 44: NaN and infinity checks to prevent gradient explosion in downstream ANN training.

This function implements the various checks and verifies the desired number of rows.

Data Volume: It verifies that the dataset has reached the required size, which is actually configured in the YAML.

NaN & Infinity Checks: In Deep Learning, a single NaN (Not a Number) or Inf (Infinity) will cause the gradient descent calculus to explode — a phenomenon known as **Gradient Explosion** — and thus destroy the entire neural network. These checks physically prevent the corrupted data and math from poisoning the further downstream stages or our network or pipeline.

Floating-Point Tolerance (`_evaluate_physical_boundaries`)

```

1 def _evaluate_physical_boundaries(self) -> None:
2     # ... hardware limits loop ...
3     lower, upper = boundaries["min"], boundaries["max"]
4     tolerance     = (upper - lower) * 0.01
5
6     breaches = (
7         (self._dataframe[parameter] < lower - tolerance) |
8         (self._dataframe[parameter] > upper + tolerance)
9     ).sum()

```

Code Listing 45: IEEE 754 floating-point tolerance guard for boundary validation.

Floating Point IEEE 754 Guard: Computers struggle with decimal calculations. For example, if our maximum temperature is 220.0, Python might return a point as 220.00000000000001. A simple less-than (<) check would see this as a problem. By dynamically calculating a 1% mathematical tolerance, we prevent the pipeline from failing due to standard computer floating-point imprecision, without compromising on catching genuine physical violations.

Surrogate Confidence Gate (`_evaluate_surrogate_confidence`)

```

1 def _evaluate_surrogate_confidence(
2     self, external_stats: Dict[str, Any]) -> None:
3     threshold_ratio = self._config.get(
4         "bal_settings", {}).get("uncertainty_threshold",
5         0.20)
6
7     for metric, stats in external_stats.items():
8         sigma          = stats.get("mean_std", 0.0)
9         lower, upper    = self.thermodynamic_bounds.get(
10             metric, (0.0, 1.0))
11         domain_range    = max(upper - lower, 1.0)

```

```
11 absolute_ceiling = threshold_ratio * domain_range
```

Code Listing 46: Dynamic uncertainty ceiling calculation to detect GPR confidence collapse.

Contextual Uncertainty Evaluation: Instead of hardcoding the uncertainty cutoff (e.g., *fail if $\sigma > 5$*), the engine calculates a **dynamic ceiling**. It calculates the total physical range of the output metric and multiplies that dynamic physical range by 20% (0.20). As soon as the GPR’s average uncertainty (σ) is larger than 20% of the entire physical domain, it triggers a “Confidence Collapse” and fails the dataset, forcing the active learning loop to acquire more data.

LangGraph Orchestration (enforce_physics)

```
1 def enforce_physics(state: DigitalTwinState) ->
    DigitalTwinState:
2     # ... state extraction and engine boot ...
3     is_compliant, report, faults = engine.execute_audit(...)
4
5     if is_compliant:
6         return {
7             "current_node":      "stage_05_physics_check",
8             "validation_passed": True,
9             "pipeline_status":   "success"    # <--- THE FINISH
10
11     }
12
```

Code Listing 47: LangGraph node interface for Stage 05 physics enforcement and pipeline termination.

The Final Gateway: It executes the audit, acting as an auditor of the final data produced, so that our ANN receives proper data and is prevented from being poisoned. If `is_compliant` is `True`, it updates the `pipeline_status` to `"success"`. This is the exact signal the LangGraph supervisor needs to permanently terminate the automated Active Learning loop and declare the Digital Twin dataset finalized.

Important Clarifications

1. Why did we hardcode the thermodynamic bounds in the Python class, but load the hardware input bounds from a YAML file?

Because they represent two fundamentally different types of constraints. Hardware inputs like print speed and configuration limits may change — we might switch the printer model in the future, so the configuration needs to be adjustable. But thermodynamic limits like the maximum theoretical tensile strength of PLA plastic are physical laws of the material, and they are not going to change. Physical laws do not change, so they are hardcoded into the engine.

2. Why do we have a specific check for Infinite (`np.isinf`) values — how could a synthetic dataset possibly generate infinity?

During Stage 04, when we are doing the Bayesian Optimization, the acquisition function attempts to maximize the Expected Improvement. If the GPR model's predicted variance near the edges of the design space becomes exactly zero, the internal division in the EI mathematical formula results in a divide-by-zero error, which produces infinity. This validation and infinity check acts as a safety net handling that divide-by-zero.

NumPy is designed to never crash if it hits a bad number. If NumPy divides by zero during the Expected Improvement calculation, it does not throw an error. It throws a tiny, easily ignored background warning (`RuntimeWarning: divide by zero encountered`), assigns the result as the mathematical concept of `np.inf` (Infinity), and just keeps running the loop.

3. What is Floating Point Tolerance and why did we take a 1% margin?

It is a safeguard against IEEE 754 floating-point arithmetic errors. The model often outputs boundary values like `220.0000000001` due to matrix multiplication imprecision. If we apply a strict mathematical inequality check, it would trigger a false-positive physical breach. Therefore, to handle this problem we dynamically calculate a 1% micro-tolerance buffer that absorbs these computational errors while still strictly catching real physical impossibilities, such as a nozzle temperature of 250°C .